

Initiativtracker

Af *Jeppe Skovgaard Andersen*

Slotshave Gymnasium

3.R

The screenshot shows a web application titled "D&D Initiative Tracker". At the top, there's a header with the title. Below it is a form to "Add Character" with fields for Name (pyrus), Initiative (18), Type (hero), and Image (pyrus.png). There are buttons for "Select Image" and "Add Character". Below the form is a turn order section titled "Turn Order (Highest Initiative First)". It shows a list of characters: vetus (Initiative: 22) - Hero, borgnog (Initiative: 16) - Enemy, grum (Initiative: 13) - Ally, and Meros III (Initiative: 11) - Enemy. To the right of the list is a circular image of a character. At the bottom, there are buttons for "Remove Selected" and "Clear All". The application is running in a browser window titled "D&D Initiative Tracker".

Applikation af: Stefan Dan Jensen og Jeppe Skovgaard Andersen

Programmering B

Mikkel Kirkeby Mosthaf

27/03/2026

Abstract

Vores formål med dette projekt var at lave en Initiativtracker som kan bruges til DND for at holde styr på hvis tur Det er indenfor en given combat. Dette løste vi ved at lave en applikation som tager info fra brugeren gemmer det i en database og videresender det tilbage til brugeren som en liste over karakterer i en runde.

Indholdsfortegnelse

Abstract	2
Opgaveformulering	3
Projektbeskrivelse.....	3
Kravspecifikation.....	3
Gruppefordelingen	3
brug af AI	4
Funktionsbeskrivelse	5
Dokumentation.....	6
Kode – eksempler	8
kode – generelt	10
Test	10
Testresultater	10
Dokumentation af AI-brug	10
Konklusion	10
Bilag.....	11

Opgaveformulering

Tema 2. Python-applikation med eget tema

I skal udvikle en python-applikation som demonstrerer brug struktureret brug af trelagsmodellen. Det vil sige at løsningen skal indeholde et data-, et logik- og et præsentationslag. Datagrundlaget og præsentationslaget er valgfrit - I bestemmer altså selv hvad løsningen handler om og hvordan den præsenteres for brugeren.

Projektbeskrivelse

Vores produkt kommer til at bestå af en "initiative tracker" til bordrollespillet "Dungeons and Dragons", det består af en applikation der kommer til at holde styr på rækkefølgen hvor ved at de forskellige spillere og/eller fjender kommer til at agere i spillet. Det kommer til at fungere som et administrativt værktøj for "game masteren"(GM) så han kan holde styr på spillets løb. Det er ofte at spillere holder styr på det ved at skrive navne ned på et papir i den rækkefølge de agere på, men fejl sker ofte og det bliver især svært når fjender ikke har rigtige navne men i stedet bare har numre(når der er flere af samme slags), hvilket er hvorfor vores applikation er nødvendigt.

Kravspecifikation

Vi kommer hovedsageligt til at fokusere på 5 krav for at sikre os at vores produkt kommer til at være både pænt, lovligt og brugbart, de krav er:

Forståelig for nybegyndere: den skal helst være simpel da det er et værktøj der har et hovedmål for dets brug og alt andet bare kommer til at være side additioner til det endelige produkt.

Let forståelig opbygning af knapper: den skal interaktivt være stærk da det er et værktøj der helst skal være så intuitivt og effektivt i de processer brugen har hentet applikationen for.

Farver der passer til vores tema: den skal helst have et design som er nemt at forstå og behageligt at kigge på, og som passer ind i den applikation som vi vil lave.

Opholdning af ophavsret: da det er en applikation baseret på et eksisterende spil, hvor rettighederne allerede er eget og de artistiske valg vi tager, kommer til at krænke ejerne af Dungeons and dragons' ophavsret.

Intet delay: vores applikation skal som sagt være simpel, det er derfor vigtigt at denne simplicitet gør at den funktion den fokuserer på, derfor kommer til at virke uden problemer eller forsinkelser.

Krav	Vurdering
Forståelig for nybegyndere	Nice-to-have
Let forståelig opbygning af knapper	Need-to-have
Farver der passer til vores tema	Nice-to-have
Opholdning af ophavsret	Need-to-have
Intet delay	Nice-to-have

Gruppefordelingen

Her kommer vi til at skrive om de fokusområder som vi hver især har inden for produktudviklingen.

Stefan kommer primært til at være vores GUI person, så han kommer til at stå for hvordan selve brugerne ser, oplever og interagerer med vores applikation.

Jeppe kommer til at være vores database person, så han kommer primært til at stå for vores database, hvordan den interagerer med brugerne og hvordan den håndterer information som den får fra brugerne

Logik laget kommer til at være fordelt ligeligt mellem de to gruppemedlemmer og tilbygninger til det laves, når brugen opstår indenfor de andre lag, for eksempel hvis databasen skal bruge kode på logiklaget for at spille sammen med GUI'en

brug af AI

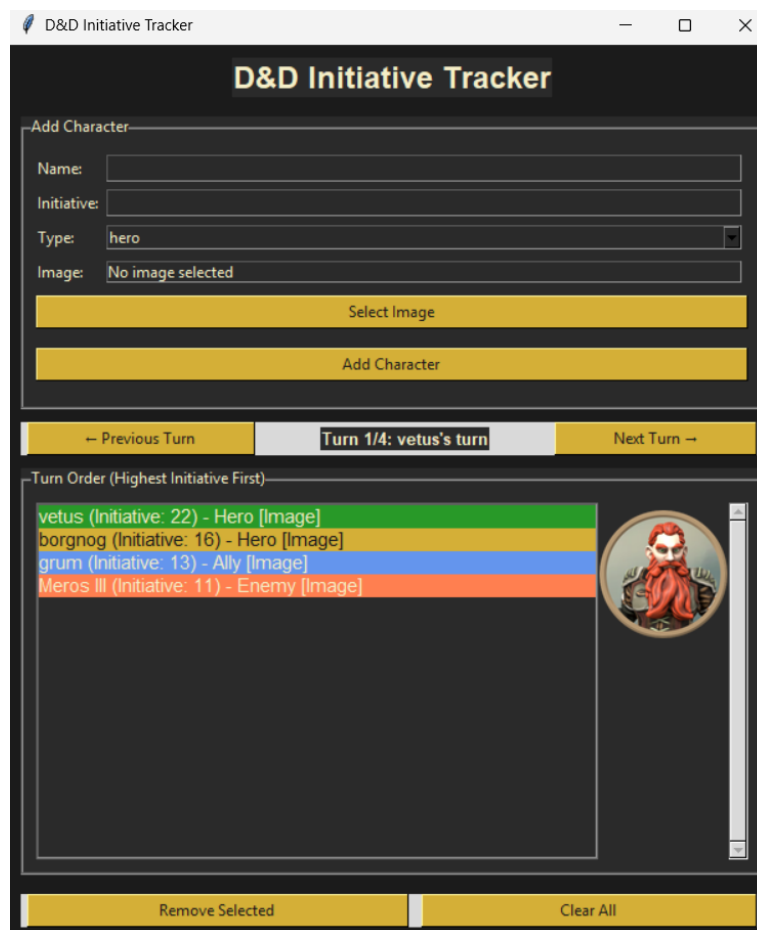
vores AI brug kommer til at bestå af hovedsageligt af hjælp til programmering af især styling og database da dette ofte sker i sprog som vi ikke har fået fordybende læring i. vi kommer til at bruge AI som et værktøj til at komme med forslag og mulige ændringer frem for at den bare laver programmet for os, dette gøres for at maksimere både læring og personligt input og præg på det endelige produkt. Vi kommer til at gøre dette ved hjælp af instruktionsfiler skrevet inde i workspacet så AI'en følger de retningslinjer vi giver den, hvilket blandt andet kommer til at indeholde dokumentation af AI-brug, som dokumenterer både prompt, svar og dato af de forskellige interaktioner vi har med AI.

Funktionsbeskrivelse

Vores kode er delt op i en række af funktion som tilsammen er der for at holde styr på hvilke karakter som har en tur i en standard runde af Dungeons & Dragons (alias: DND/D&D) Combat. De forskellige funktioner består af, en funktion til at lave nye entiteter, som lader en både bestemme navn og initiativ på de forskellige entiteter. Vi har også lavet en funktioner som lader en tilskrive en status til en entitet som enten kan vager Hero, Ally and Enemy. Udover dette har vi også en displayliste med entiteternes information. Alle disse funktioner virker ved at kalde og have et samspil med vores database til lagring af datasættet over vores entiteter.

Funktionerne som står få at man kan navigere i vores program, er delt op i 4 forskelle knapper, som er "next turn", "previous turn", "remove selected" og "clear all". Vores knapper med "turn" i navnet er der så man kan skifte mellem hvis tur det er på nuværende tidspunkt. Knapperne "remove selected" og "clear all" er der for at redigere i vores datasæt, hvor "remove selected" kun fjerner en udvalgt entitet fra datasættet. i modsætning fjerner "clear all" alle entiteter i vores datasæt, da dette kan være Irriterende hvis man kommer til at trykke på den ved en fejl, har vi gjort at den spørger en om man er sikker enden den fuldføre funktionen.

Funktionerne som er der få at man kan tilføje nye entiteter er knappen "Add Character". Denne Funktion tager informationen fra som bliver sat ind i tekst felterne Name og Initiative, drop down menuen og imagevælgeren. Og for dem til at blive en del af Vores database hvor den så tager information fra databasen til at lave Vores "Turn Order"



Figur 1, applikation i brug

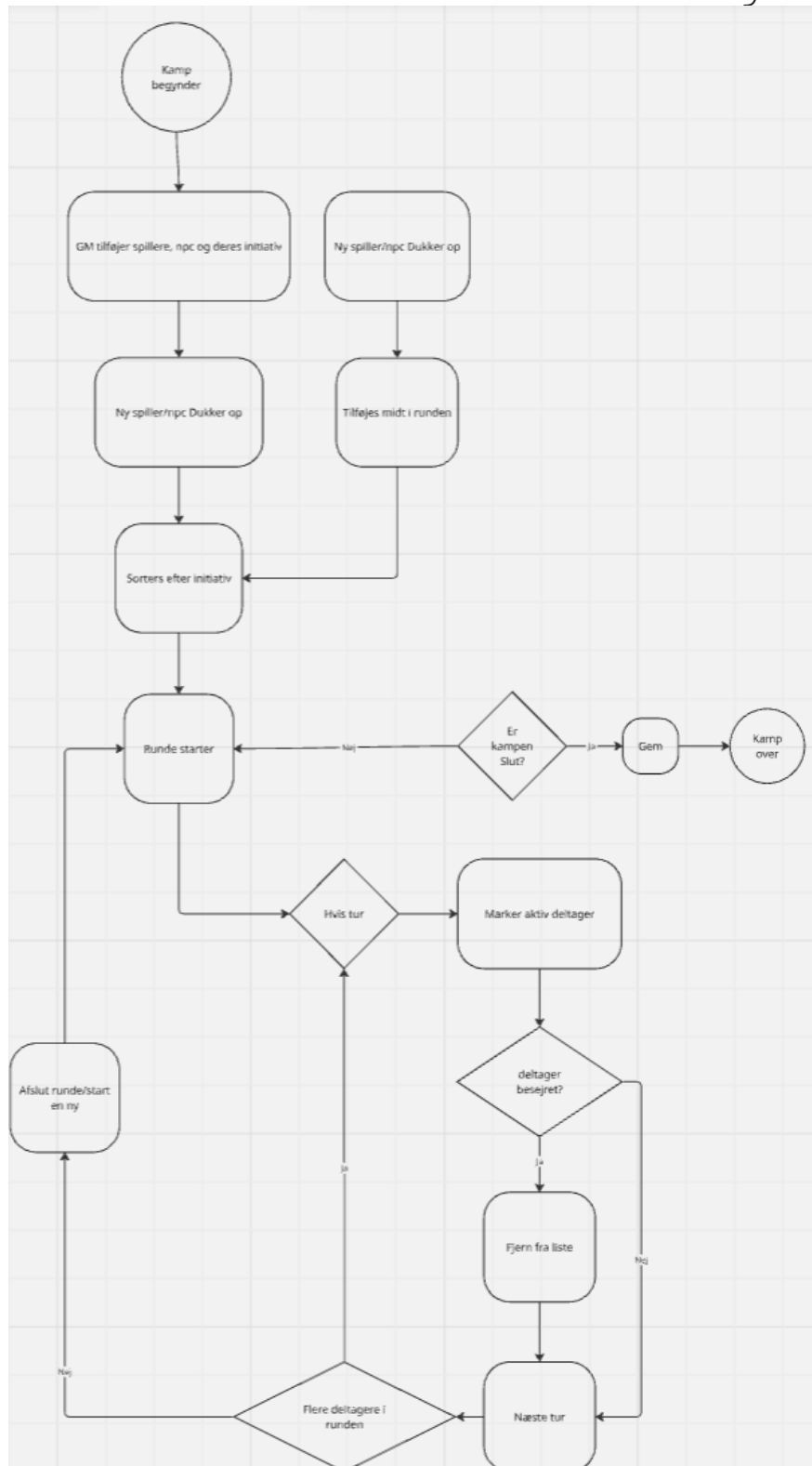
Dokumentation

Vi har valgt at opbygge vores program Primært med trelags modellen, det betyder at strukturen er opbygget hvor de forskellige dele fordeles over deres forskellige funktioner. Dette har vi gjort for at skabe overblik over vores kode og så vi kunne arbejde på forskellige dele på samme tid uden at påvirke hvad den anden person havde gang i. Programmet består af tre dele som er databaselaget, brugeroverfladen laget og et logi lag.

Databaselaget ligger i mappen DB som Består af både selve databasen og den Python kode som håndterer læring og hentning af data. Det er her som vi opbevarer alt informationen om vores entiteter og hvor selve funktionerne for at oprette, hente og slette karakterer bliver udført. Brugeroverflade laget lægger en mappe ved navn GUI Som består af en HTML fil som er der for at skabe struktur og en Python fil for at håndterer brugernes input. Vi har også en mappe som hedder statik som indeholder information som billederne til vores karakterer da dette ikke kunne indsættes til eller gemmes i den database som vi er blevet oplært i.

Hele programmet skal køres igennem vores fil Main.py. Den fil virker lidt som en tragte for alt informationen og alle programmerne, hvorfor alt informationen og programmerne til at arbejde sammen. Vi har også en separat fil til data og datastrukturer som bliver brugt i programmet. Overordnet virker programmet ved at brugeren hans regering med brugergrænsefladen, Hvor python koden så tager informationen og få sendt den videre til databasen for at gemme dataen, og så tager jeg koden dataen fra databasen tilbage ud til brugergrænsefladen for at vise brugeren deres data.

Vi har også både instruktioner og logfiler til brugen af i det det gør at de overholder de rammer som er opsat for Ai brug og for at overholde de realistiske rammer for egen forståelse af faget og programmet.



Figur 2 Flowchart lavet i Miro over brug af applikation

Kode – eksempler

Her gennemgår jeg et par eksempler på den kode som jeg brugte til at få dataene registret og gemt

tilføje en karakter

```
def add_character(name, initiative, status, pic):
    """
    Add a new character to the database.

    Args:
        name (str): Character name
        initiative (int): Initiative value (higher = acts first)
        status (str): Character type ("hero", "ally", or "enemy")
        pic (str): Picture path (optional)

    Returns:
        bool: True if successful, False if it fails
    """
    try:
        with sqlite3.connect(DB) as conn:
            cursor = conn.cursor()
            cursor.execute(
                "INSERT INTO initiativefordnd (name, initiative, status, pic)
VALUES (?, ?, ?, ?)",
                (name, str(initiative), status, pic)
            )
            conn.commit()
        return True
    except Exception as error:
        print(f"Error adding character: {error}")
        return False
```

denne kode er til for at tilføj en karakter til vores database. den tager information fra testfelterne og drop down menuen, og sætter dem ind som data til Vores database.

```
def add_character(self, name, initiative, char_type="hero", pic_path=""):
    # Check if character already exists
    if db.character_exists(name):
        return False

    # Add to database
    success = db.add_character(name, initiative, char_type, pic_path)

    if success:
        # Reload from database to stay in sync
        self._load_from_database()
        return True

    return False
```

her har vi koden om en karakter kan blive tilføjet til applikationen, den tjekker om det er en karakter som har samme navn allerede, er der, vis dette er sandt ville det skabe en error, men vis en karakter ved samme navn ikke eksister så tilføjede den karakter til vores database

Slet en karakter

```
def delete_character(name):
    """
    Delete a character from the database.

    Args:
        name (str): The name of the character to delete

    Returns:
        bool: True if deleted successfully, False otherwise
    """
    try:
        with sqlite3.connect(DB) as conn:
            cursor = conn.cursor()
            cursor.execute("DELETE FROM initiativefordnd WHERE name = ?",
                           (name,))
            conn.commit()
            return cursor.rowcount > 0
    except Exception as error:
        print(f"Error deleting character: {error}")
        return False
```

Her har vi lavet en kode der kun er der for slette karakterer fra databasen det vil sige at den reagerer på Knappen "remove selected" og fjerner den valgte karakter.

kode – generelt

mit fokus på databaselaget er vores applikation er en god del af programmering foregået i filen Det er vi gode, Det er det dokument fokuserer primært på hvordan Det er at resten af Vores kode fungerer sammen med den database som var SQLite. Jeg arbejdede også lidt med hvordan Det var at vi fik inputtet fra brugerne ind til Vores kode som samlede det hele sammen i en database fil.

Test

Testen som jeg lavede hedder en "unittest", den opererer inden for meget simple termer og en måde for at tjekke om en funktions output giver de rigtige resultater. Jeg satte min test op så den kunne tjekke om alle mine funktioner indenfor DB kode kom ud med et resultat som var som kan bruges indenfor Vores kode, Jeg testede kodet ind for at den kunne lave ny database hvis der manglede en, Sådan kunne få karakterer, opdater initiativet, se om Der er en karakter ved samme navn som allerede eksisterer, om den kunne slette enkelte karakterer og om den kunne slette alle karakterer.

Testresultater

Da jeg kørte testen igennem pytest Og den viste at alle de forskellige funktioner som bliver testet kom ud med at have bestået de test som par i test lavede på den (3).

Dokumentation af AI-brug

Igennem vores projekt har vi brugt AI både konstruktivt og undersøgende til bearbejdning af kode og fejlfinding, dette gøres ved fremfor at få AI til bare at lave vores kode for os, så beder vi den om at komme med eksempler på løsningsmetoder og kode strategier, såsom hvor det vil være bedst at bruge statemachine programmering eller OOP. For at dokumentere dette har vi lavet et "AI-guideline" dokument der tvinger AI agenten til at følge en række retningslinjer der gør så den arbejder på et niveau jeg kan både forstå og lære af og lader mig bruge AI som et værktøj fremfor en krykke. Iblandt disse retningslinjer var der også kravet om en "AI-log" hvor der er dokumenteret dag, prompt og resultat for hver interaktion med den. Begge disse dokumenter ligger i vores repository. Vi har dog også forholdt os meget kritisk over den hjælp den giver, da AI ikke er perfekt og kan misforstå vores hensigter og spørgsmål. Vi brugte dog AI til at skrive vores testdokument fra pytest, da vores undervisning og forståelse for unittests ikke var tilstrækkelig til ordentligt at teste vores projekt

Konklusion

I dette projekt har vi udviklet en Python applikation i form af en initiative tracker til Dungeons and Dragons med fokus på struktureret programmering og brug af trelagsmodellen. Applikationen løser et konkret problem som ofte opstår under spillet nemlig at holde styr på turrækkefølgen mellem spillere og fjender på en overskuelig måde. Ved at gøre denne proces til en digitale applikation kræver det ikke længere at ham som holder styr på spillet har behov for at huske et papir eller huske hvad rækkefølgen var til næste gang Det kan i stedet for bare havde den her applikation som gemmer deres rækkefølge til næste gang.

Projekt projektet forløbet op til Vores krav som vi opstillede i starten, så nye kan bruge vores applikation. Vi har også opbygget knapper med tekst på i stedet for ikoner så det er nemt for folk at forstå hvad de gør.

Arbejdsprocessen har givet en bedre forståelse af hvordan trelagsmodellen virker Når man skal programmere

overordnet har projektet opfyldt Vores krav og Vi har demonstreret en funktionel Python applikation som er opbygget med trelagsmodellen, Processen har styrket både min tekniske færdigheder inden for programmering og har givet mig en forståelse indenfor hvordan programmer skal struktureres

Bilag

```
PS C:\Users\Jeppe\Desktop\Code\Eksamen\GitForEksamenOpgave> python -m pytest -v tests
===== test session starts =====
platform win32 -- Python 3.13.12, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Jeppe\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Jeppe\Desktop\Code\Eksamen\GitForEksamenOpgave
collected 6 items

tests/test_dbcode.py::test_init_db_creates_table PASSED [ 16%]
tests/test_dbcode.py::test_add_get_character PASSED [ 33%]
tests/test_dbcode.py::test_update_initiative PASSED [ 50%]
tests/test_dbcode.py::test_character_exists PASSED [ 66%]
tests/test_dbcode.py::test_delete_character PASSED [ 83%]
tests/test_dbcode.py::test_del_all_characters PASSED [100%]

===== 6 passed in 0.13s =====
PS C:\Users\Jeppe\Desktop\Code\Eksamen\GitForEksamenOpgave>
```

Figur 3 DB pytest